Lecture 10: Customizing constructors for correctness and convenience

Reasons for customized constructors, designs for custom constructors

Video Lessons

- Lesson 1 - Ways of using constructors
- Lesson 2 - Exceptions

Motivation

Up to now, whenever we design a class, we immediately write a constructor for that class whose implementation is very formulaic:

- The constructor takes one parameter for each field defined in the class, with the same name as the field.
- The constructor takes one parameter for each field inherited from any base classes, (usually) with the same name as the field.
- If the constructor derives from some base class, the *first* line of the constructor invokes the `super` constructor with any parameters needed by the base class' constructor.
- The rest of the constructor is a list of *initializer statements*, `this.field = param`, that initialize each field of this class with the relevant parameter.

But consider the downsides of this approach. The user of a class must remember *all* of the fields, *all* of the time, even when they may not all be relevant. And worse, perhaps it doesn't make sense to construct objects with every possible value that might be allowed by the types of the fields. How can we fix these problems?

10.1 Constructors with default options

Suppose you were implementing a game of Tetris, with new pieces appearing at the top of the 30-row screen. For simplicity, assume for now that there are just 2x2 boxes

and L-shaped pieces.

> ***Do Now!***
>
> How might you represent this?

In *general*, each piece would need to record its horizontal and vertical position, since that could change over the course of the game. So we might begin as follows:

```
// To represent a Tetris piece
interface ITetrisPiece { ... }

// To share implementations common to all Tetris pieces
abstract class ATetrisPiece implements ITetrisPiece {
  int xPos;
  int yPos;
  ATetrisPiece(int x, int y) {
    this.xPos = x;
    this.yPos = y;
  }
}
// To represent a 2x2 square Tetris piece
class Square extends ATetrisPiece {
  ...
  Square(int topLeftX, int topLeftY, ...) {
    super(topLeftX, topLeftY);
    ...
  }
}
// To represent an L-shaped Tetris piece
class LShape extends ATetrisPiece {
  ...
  LShape(int cornerX, int cornerY, ...) {
    super(cornerX, cornerY);
    ...
  }
}
```

When writing your tests, you would certainly construct pieces at arbitrary y coordinates: 0, negative, between 0 and the screen height (30), or greater than the screen height. But in the main code for your game, you would *always* construct pieces at the height of the screen. So why bother forcing the user to specify it each time?

We can remedy this by defining a *convenience constructor* whose job is to construct our object with some reasonable *default values* for some of the fields.

```
abstract class ATetrisPiece implements ITetrisPiece {
  ...
```

```
  ATetrisPiece(int x, int y) {
    this.xPos = x;
    this.yPos = y;
  }
  // NEW CONSTRUCTOR
  ATetrisPiece(int x) {
    this.xPos = x;
    this.yPos = 30; // screen height
  }
}
class Square extends ATetrisPiece {
  ...
  Square(int topLeftX, int topLeftY, ...) {
    super(topLeftX, topLeftY);
    ...
  }
  // NEW CONSTRUCTOR
  Square(int topLeftX, ...) {
    super(topLeftX);
    ...
  }
}
```

We have defined a new constructor in our base class that takes *only one parameter*, and supplies a default value for the other one. And we have defined a new constructor in our derived class that takes one fewer parameter, and calls the convenience constructor on the super class accordingly.

> ***Do Now!***
>
> Why should the second constructor for `Square` not try to make up its own value for `topLeftY`?

We can now construct examples of `Square`s:

```
// In some ExamplesTetris class
// Calls the first constructor, and creates a square at position (3, 15)
ITetrisPiece square1 = new Square(3, 15, ...);
// Calls the second constructor, and creates a square at position (3, 30)
ITetrisPiece square2 = new Square(3, ...);
```

Defining two constructors for the same class, but with different signatures (i.e., the number and types of parameters), is called *overloading*. Java determines which constructor to invoke by examining the types of the arguments passed in the call. This is *very* different from dynamic dispatch: Java can determine *statically* which constructor to call just by looking at the static types of the parameters, rather than by waiting until runtime to figure out what kind of object is being used to invoke a method.

It is prohibited to define two constructors with the same number and type of parameters (or related parameters, like `int` and `double`).

> ***Do Now!***
>
> Why?

It is also possible to define several methods with the same name but different signatures, and this too is called overloading. We have already seen examples of this, without realizing it: the `checkExpect` method in the `Tester` class can take two `int`s, or it can take two `boolean`s, or it can take two `String`s, etc., but it can never take values of two different types. This is because the `checkExpect` is overloaded with several definitions, each of which takes two parameters of the same type, but where each definition is different from all the others.

Interlude 1: invoking one constructor from another

Look at the two constructors for `ATetrisPiece`. They both initialize all the fields of the constructor, with nearly identical code. And as we'll see below in Combining convenience with correctness, our constructors may get far more complicated; it would be very nice to avoid such repetition.

In fact, we'll elevate that desire to a design principle: There should be only one place in the code that does any particular task, known as a "single point of control".

With inheritance, we saw how `super` let us invoke a constructor on the base class to let it initialize all the inherited fields for us. Here, we have no superclass; we want to invoke a *different* constructor on *this* class instead. We can write that as follows:

```
abstract class ATetrisPiece implements ITetrisPiece {
  ...
  ATetrisPiece(int x, int y) {
    this.xPos = x;
    this.yPos = y;
  }
  // NEW CONSTRUCTOR
  ATetrisPiece(int x) {
    // invokes the other constructor
    this(x, 30); // screen height
  }
}
```

Here we use the keyword `this` not to refer to this object, per se, but rather to the constructors of this object.

> It's almost as if we're *overloading* the `this` keyword to have two different meanings...

Just as invoking a `super(...)` constructor must be the very first line of a constructor, so too invoking another constructor for this class via `this(...)` must be the very first line of a constructor.

10.2 Interlude 2: defining constants in Java

We have said several times in this course that "Java has no functions, unlike DrRacket", and this is true, but the more general complaint is that "Java does not have `(define ...)`, unlike DrRacket." And this lack of `(define ...)` means it is slightly inconvenient to define constants in Java, too.

Here is one mechanism for defining constants in Java. (There are other, better ways, but they involve several additional keywords whose meanings are beyond the scope of this course.) Our naming convention will be that all constants are written entirely in uppercase letters, with words separated by underscores. Within an interface, we simply define a field and initialize it with a value. Then code inside any class that implements that interface can use that name as a constant. *Note:* code in any class that does *not* implement that interface *cannot* use that name, as it is not in scope.

We could rewrite our example constructor above as follows:

```
interface ITetrisPiece {
  int SCREEN_HEIGHT = 30;
}

abstract class ATetrisPiece implements ITetrisPiece {
  ...
  ATetrisPiece(int x, int y) {
    this.xPos = x;
    this.yPos = y;
  }
  ATetrisPiece(int x) {
    this(x, SCREEN_HEIGHT);
  }
}
```

Here we take advantage of the new constant we defined (which is available to us because `ATetrisPiece implements ITetrisPiece`) and the `this(...)` notation to write our convenience constructor very concisely, with no repetitions or distractions. How convenient!

10.3 Constructors that enforce data integrity: Exceptions

Consider now a completely different example: suppose we want to represent a date, consisting of three integers representing a day, a month, and a year.

> ***Do Now!***
>
> With the tools we have so far, define this `Date` class.

These types help catch some errors statically: we can't mistakenly construct a `new Date("January", 30, 2022)`, since we said the month must be an integer. But sometimes the types are not enough to capture the full meaning of data and the restrictions on what values can be used to initialize different fields. We know that the month must be between 1 and 12, and the day must be between 1 and 31 (though there are additional restrictions on the day, depending on the month and whether we are in a leap year). We might also consider restricting the year to the range 1500—2100, to describe (semi-)recent history or near-future.

Let's make the following Date examples:

```
// Good dates
Date d20100228 = new Date(2010, 2, 28);   // Feb 28, 2010
Date d20091012 = new Date(2009, 10, 12);  // Oct 12, 2009

// Bad date
Date dn303323 = new Date(-30, 33, 23); // ???
```

Of course, the third example is just nonsense. While complete validation of dates (months, leap-years, etc...) is a study topic in itself, for the purposes of practicing constructors, we will simply make sure that the month is between 1 and 12, the day is between 1 and 31, and the year is between 1500 and 2100.

```
Date(int year, int month, int day) {
  if(year >= 1500 && year <= 2100) {
    this.year = year;
  }
  else {
    ???
  }

  if(month >= 1 && month <= 12) {
    this.month = month;
  }
  else {
    ???
  }

  if(day >= 1 && day <= 31) {
    this.day = day;
```

```
    }
    else {
      ???
    }
  }
```

What should we do with all the question-marks? In those cases, the supposed `Date` makes no sense. Our program ought to throw up its hands and give up: it *makes no sense to continue executing*. To do this, we need a new language feature: *exceptions*. Instead of *returning* from a method, or finishing the constructor, we want to throw away what we're doing and terminate the program with a helpful error message:

```
// In class Date
Date(int year, int month, int day) {
  if(year >= 1500 && year <= 2100) {
    this.year = year;
  }
  else {
    throw new IllegalArgumentException("Invalid year: " + Integer.toString(year));
  }

  if(month >= 1 && month <= 12) {
    this.month = month;
  }
  else {
    throw new IllegalArgumentException("Invalid month: " + Integer.toString(month));
  }

  if(day >= 1 && day <= 31) {
    this.day = day;
  }
  else {
    throw new IllegalArgumentException("Invalid day: " + Integer.toString(day));
  }
}
```

Here, we are throwing an `IllegalArgumentException` object. The `IllegalArgumentException` class is a subclass of `RuntimeException`, which is a subclass of the `Exception` class. From their names, you can guess their purpose statements:

- The `Exception` class is the abstract base class of all exceptions. (It doesn't follow our naming convention of starting with an 'A', though.)
- The `RuntimeException` class is the base class for all exceptions that happen at runtime. It is a kind of exception, so it is-a (i.e., it inherits from) `Exception`. (Another runtime exception you might have seen is a `NullPointerException`.)
- The `IllegalArgumentException` class represents the mistake of passing an illegal

argument to a constructor or method.

If the program ever executes a statement like:

```
throw new ???Exception("... message ...");
```

Java stops the program and signals the *error* through the constructed instance of the *???Exception* (where the `???` are replaced by the name of whichever particular exception is desired). For our purposes, this will simply terminate the program and print the given error message.

10.4 Interlude 3: removing redundancy with a utility class

Notice how repetitive the constructor above is: it checks three numbers for whether they lie within a range, and if not, throws an exception. Perhaps we could abstract away this repetition (along the lines of Lecture 9)?

> ***Do Now!***
>
> Design a method `checkRange`, that takes a number to test, a minimum and maximum value for the range, and an error message to throw.

```
int checkRange(int val, int min, int max, String msg) {
  if (val >= min && val <= max) {
    return val;
  }
  else {
    throw new IllegalArgumentException(msg);
  }
}
```

What class does this code belong in? This code truly is a *function* — it makes no mention of `this`, and really doesn't depend on any object at all. But Java doesn't have functions, so we have to stick it inside *some* class.

When faced with this situation, developers often write a *utility class*, that acts as a container for these functions that don't have any other class they really need to be part of:

> In subsequent courses, you'll learn about additional keywords in Java that can improve this code further, and make it even easier to use.

```
class Utils {
  int checkRange(int val, int min, int max, String msg) {
    if (val >= min && val <= max) {
```

```
      return val;
    }
    else {
      throw new IllegalArgumentException(msg);
    }
  }
}
```

Now we can rewrite our constructor to use this helper method from the utility class:

```
// In class Date
Date(int year, int month, int day) {
  this.year = new Utils().checkRange(year, 1500, 2100,
    "Invalid year: " + Integer.toString(year));
  this.month = new Utils().checkRange(month, 1, 12,
    "Invalid month " + Integer.toString(month));
  this.day = new Utils().checkRange(day, 1, 31,
    "Invalid day: " + Integer.toString(day));
}
```

Much shorter...and clearer to read, too!

Notice that now our constructor never explicitly throws an exception—that's been moved into the helper method. But the meaning of the program hasn't changed: if the `Tester` class' `checkRange` method throws an exception, it will abort the constructor too, along with the rest of the program, and still raise the same error as before.

10.5 Testing exceptions in constructors

If we have a new language feature, we must be able to test that it works correctly. The *tester* library provides methods to test constructors that should throw exceptions:

```
// In Tester
boolean checkConstructorException(Exception e,
                                  String className,
                                  ... constr args ...);
```

For example, the following test case verifies that our constructor throws the correct exception with the expected message, if the supplied year is `53000`:

> Don't worry for now about how providing the name of a class is sufficient to allow the tester library to construct your class. The tester library uses many advanced features of Java to make it easier to write tests; we'll encounter and explain some of them later in the course.
>
> Another of those advanced features is visible here too: notice that the tester library can test for the presence of an error, *and continue running subsequent tests*. Other than the tester library, no code that we write in this course can do this.

```
t.checkConstructorException(
     // the expected exception
     new IllegalArgumentException("Invalid year: 53000"),

     // the *name* of the class (as a String) whose constructor we invoke
     "Date",

     // the arguments for the constructor
     53000, 12, 30);
```

Run your program with this test. Now change the test by providing an incorrect message, incorrect exception (e.g. `NoSuchElementException`), or by supplying arguments that do not cause an error, and see that the test(s) fail.

> **Exercise**
>
> Explore the documentation for the tester library, and see how to test for exceptions raised by *methods*. Write a test for `checkRange` that checks for this exceptional behavior.

10.6 Combining convenience with correctness

Suppose we want to define a convenience constructor for `Date` that accepts only a month and a day, and assumes the current year. Should we copy and paste all the error-handling code we wrote in the constructor above for months and days? No! We can use the `this(...)` notation to delegate to the constructor that does the error checking for us:

```
// In class Date
Date(int month, int day) {
  this(2022, month, day);
}
```

Now, trying to create `new Date(14, -30)` will raise the same exception complaining about invalid months as would `new Date(2022, 14, -30)`.